

Project Name: Sudoku

Lusine Torikyan

Program overview

The project is a Sudoku game. Sudoku is a number-placement puzzle, where each row, column, and 3x3 subgrid must contain digits from 1 to 9 exactly once, and the sum of the numbers has to be 45. The project is built using Python's Tkinter library for the graphical user interface. The project demonstrates our understanding of fundamental programming concepts such as functions, loops, conditionals, and event-driven programming. The main idea of the project is to create a playable puzzle for the user by removing numbers based on the selected difficulty level.

Sudoku Generation

The program begins by generating a fully solved 9x9 Sudoku grid based on pattern based Latin Square Sudoku generator algorithm. The puzzle is guaranteed to have one unique solution, and to be solvable, the required minimum number of clues is 17

(https://www.researchgate.net/publication/51978217_There_Is_No_16-Clue_Sudoku_Solving_the_Sudoku_Minimum_Number_of_Clues_Problem_via_Hitting_Set_Enumeration)

Making the Sudoku Puzzle the user is going to play

After generating a complete Sudoku, the program removes numbers from the grid based on the chosen difficulty level: Easy - 2, Medium - 3, Hard - 5 numbers from each row. The rule of the puzzle to be solvable and have a unique solution is maintained.

Graphical Interface

Used Tkinter library to build an interactive graphical interface. The GUI allows the user to:

1. Start the program using "Start" button
2. Choose the difficulty level using Easy, Medium, or Hard buttons
3. To play the game by creating the Sudoku grid, making the cells editable, allowing the user to check the answers by clicking the "Enter" button on their keyboard.
4. Sending messages/output to the user by *messagebox*

Playing Sudoku

- The user plays the Sudoku by inputting numbers using their computer keyboard.
- Every time the user enters a number and presses Enter on their keyboard, the program checks the input.

1. If contains the user's inputted number with the correct number from the original solved Sudoku,
2. If the number is correct, the number becomes green, if not, stays blue.

3. The user has 3 mistake opportunities. If the number is wrong, it counts toward mistake opportunities decline.
4. If the user leaves a cell blank, or types invalid characters(letters, symbols...), the program treats it as incorrect input as well.
5. If the user accumulates 3 mistakes, the program ends the game and displays “Game over, you have made 3 mistakes”.

Winning Condition

If the user manages to complete the puzzle without exceeding the error limit, the program shows “Congratulations!", "You solved the Sudoku! Saving automatically...”

File Handling

The computer saves the Sudoku grid with its solutions in a *.json* file, and it reads it from the file only after the user has successfully solved the Sudoku.

The code is organized in the following modules:

main.py connects everything and runs the code

generate_sudoku.py generates the Sudoku with unique solution based on pattern based Latin Square Sudoku generator algorithm

start_difficulty.py responsible for the graphical interface(Start, Difficulty levels, Sudoku grid). Handles user input / input checking. Winning conditions, messages for the user.